AGENTIC AI WORKSHOP · DAY 1 OF 10

What "agentic" actually means

Classical ML vs GenAI vs agents, the autonomy spectrum, and your first live calls to Claude and a local model.

You build: **First call** · about 1.9 h of material

Contents

1. A chatbot talks. An agent acts. — Why this matters · 5 min
2. Classical AI → Generative AI → Agentic AI — Theory · 10 min
3. How much should an agent decide on its own? — Theory · 8 min
4. The three building blocks — Theory · 7 min
5. Lab 1: Set up your laptop — Lab · beginner · 20 min
6. Lab 2: Your first call to Claude — Lab · beginner · 15 min
7. Lab 3: Claude vs a local model — Lab · beginner · 15 min
8. Route by uncertainty — Theory · 10 min
9. Lab 4: Is this even an agent? — Lab · intermediate · 15 min
10. Check yourself — Quiz · 5 min
11. Day 1 recap — Recap · 3 min
12. Quiz answer key

WHY THIS MATTERS 5 MIN

A chatbot talks. An agent acts.

Everyone says "agentic". By the end of today you'll be able to tell, from a transcript, whether something really is one.

Northstar Pay's risk team asks: **"Is ACME over its processing limit this month?"**

Ask a plain chatbot and it will **invent a confident number**: it has no idea what ACME processed today.

A human analyst would open ACME's merchant file, get today's EUR→USD rate, add everything up in USD and compare it with the limit. **An agent does exactly that:** it decides on a step, runs a tool, looks at the result and decides the next step. That's what you'll build this course: **Meridian**, the merchant-risk agent at Northstar Pay (a fictional payment processor).

By the end of today you can

- Explain the difference between classical ML, generative AI and agentic AI
- Place a system on the autonomy spectrum
- Name the three building blocks every agent framework uses
- Run Claude (via OpenRouter) and a local Ollama model from Python
- Check a transcript and say whether it's really an agent

THEORY 10 MIN

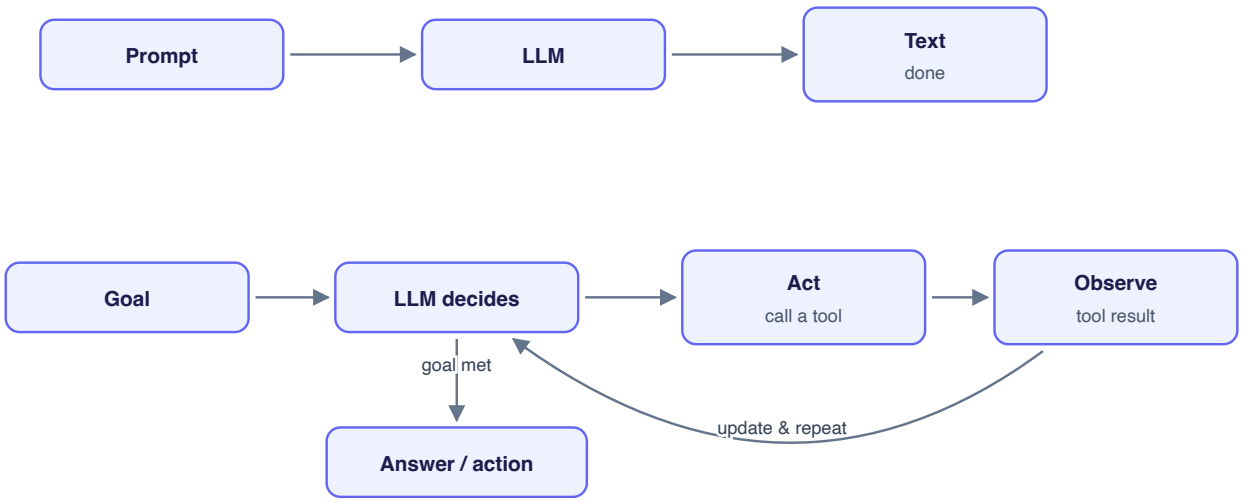
Classical AI → Generative AI → Agentic AI

The jump to "agentic" isn't better text. It's a closed loop of action.

	Classical ML	Generative AI	Agentic AI
What it is	a model trained for **one** prediction	a model that **generates** content from a prompt	a model in a **loop** with **tools** and a **goal**
Northstar example	fraud score: transaction → 0.87	"explain what a volume spike means"	look up volume, convert FX, compare with limit, raise an alert
Knowledge	learned from training data	frozen at training time	**fetched live** with tools
Who decides the steps?	the engineer	nobody: one shot	**the model**, inside limits set by code
When it fails	wrong label	wrong text (hallucination)	wrong **action**: higher stakes

The diagram shows the difference, step by step:

Open loop (GenAI) vs closed loop (agent)



1. Generative AI is an open loop: a prompt goes in, text comes out, and it's finished.
2. An agent starts from a goal, e.g. 'Is ACME over its processing limit?'. The model decides what to do first.
3. It ACTS: it asks your code to run a tool, e.g. look up ACME's merchant file.
4. It OBSERVES the result: two batches are in EUR, so it needs an FX rate.
5. The observation changes the next decision. This loop is what makes it an agent.
6. When the goal is met, it answers (9.92M USD vs 9.5M → OVER_LIMIT) or takes an action.

Key points

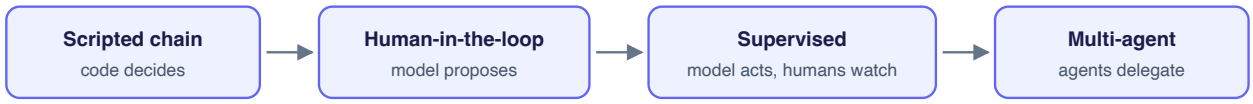
- Agentic = act → observe → decide again, until the goal is met.
- Agents don't replace ML models: Meridian can call the fraud model as one of its tools.
- Higher capability brings higher stakes: an agent can do the wrong thing, not just say it.

THEORY 8 MIN

How much should an agent decide on its own?

Autonomy is a dial, and in payments you set it per action.

The autonomy spectrum



1. Scripted chain: fixed steps written in code. Example: classify every merchant support email. (Day 3)
2. Human-in-the-loop: Meridian proposes a processing-limit increase; a risk officer approves it. (Day 4)
3. Supervised autonomy: Meridian answers ops questions alone but is blocked from changing limits. (Days 8–9)
4. Multi-agent: research, writer and critic agents produce the monthly risk report together. (Day 7)
5. Autonomy is decided per ACTION, not per system: reading data can be autonomous; moving money needs approval.

Rule of thumb for payments: the more an action costs if it goes wrong, and the harder it is to undo, the more a human should be involved. Reading data can be fully autonomous. Raising a limit, issuing a refund or releasing a reserve needs approval.

Key points

- The same agent can be autonomous for questions and human-approved for actions.
- You'll build each level during the course: chain (Day 3), approval gate (Day 4), crew (Day 7), guardrails (Days 8–9).

THEORY 7 MIN

The three building blocks

Every agent framework is assembled from the same three pieces. Learn them once and frameworks become syntax.

1. Tool call: from thinking to doing

The model returns a **structured request** such as `{"name": "lookup_merchant", "input": {"merchant_id": "ACME"}}`. **Your code** runs it and returns the result. The model never executes anything itself.

2. Memory: what it knows beyond this message

Kind	Lasts for	Example
Working	one run	the FX rate fetched two steps ago
Session	one conversation	"we were talking about ACME"

Kind	Lasts for	Example
Long-term	across sessions	"this analyst prefers GBP figures"
Knowledge	while documents are current	Northstar's chargeback policy (Day 5)

3. Planning: deciding the sequence

Pick the next step each turn (ReAct), follow an explicit checklist (plan-and-execute), or critique and retry (reflection). Day 2 builds all three.

> **Agent = Model + Loop + { Tools, Memory, Planning } + Guardrails**

LAB BEGINNER 20 MIN

Lab 1: Set up your laptop

Python, a virtual environment and an OpenRouter key: the same setup for the whole course.

Goal. A working Python 3.13 environment with the course code and your API key in place.

1. Install Python 3.13

Download the installer from **python.org** (Mac or Windows). On Windows tick **"Add python.exe to PATH"**. Full walkthrough: Set up your laptop.

2. Create the course environment

RUN

```
CD <course-folder>
python3.13 -m venv .venv          # WINDOWS: PY -3.13 -M VENV .VENV
SOURCE .venv/bin/activate         # WINDOWS: .VENV\SCRIPTS\ACTIVATE.PS1
```

3. Install Day 1 packages

RUN

```
CD day1-foundations/code
pip install -r requirements.txt
```

4. Get an OpenRouter key

openrouter.ai → **Keys** → **Create Key** (set a credit limit). It starts with `sk-or-v1-`. Treat it like a password.

5. Create your .env file

Open `.env` and paste your key after `OPENROUTER_API_KEY=`.

RUN

```
CP .env.example .ENV              # WINDOWS: COPY .ENV.EXAMPLE .ENV
```

Expected result. Your prompt starts with `(.venv)` and `python --version` prints **3.13.x**.

LAB BEGINNER 15 MIN

Lab 2: Your first call to Claude

One request, one response, and a look inside it. Day 2's agent loop is built from exactly these fields.

Goal. Call Claude through OpenRouter and understand every part of the response.

1. Meet llm.py: the only shared file

It points the official Anthropic SDK at OpenRouter. Every lab imports `client` and `MODEL` from it.

`llm.py`

```
"""llm.py: connects to Claude through OpenRouter. Every lab imports `client` and `MODEL` from here."""
import os

import anthropic
from dotenv import load_dotenv

load_dotenv()  # reads OPENROUTER_API_KEY (and optional MODEL) from .env

# The official Anthropic SDK, pointed at OpenRouter's Anthropic-compatible endpoint
client = anthropic.Anthropic(
    base_url=os.getenv("BASE_URL", "https://openrouter.ai/api"),
    auth_token=os.environ["OPENROUTER_API_KEY"],
)
MODEL = os.getenv("MODEL", "anthropic/claude-opus-5.5")
```

2. Run the first call

RUN `python lab2_first_call.py`

3. Read the response anatomy

`content` is a **list of blocks**, not a string. `stop_reason` says why it stopped: `end_turn` = finished, `tool_use` = it wants to act (Day 2). `usage` gives tokens, which means cost.

4. See that the model has no memory

The memory test works only because **we** send the whole conversation back every time. That's the memory building block at its simplest.

Expected result. Three bullet points about chargebacks, then `stop_reason: end_turn`, the block types and the token counts.

Full file

`lab2_first_call.py`

```
"""Day 1 - Lab 2: your first call to Claude (through OpenRouter).

Run:  python lab2_first_call.py
"""
from llm import MODEL, client   # Claude via OpenRouter (see llm.py)

# 1) One request, one response
response = client.messages.create(
```

```
    model=MODEL,
    max_tokens=1000,
    system="You are a payments operations assistant at Northstar Pay. Be concise.",
    messages=[{"role": "user", "content": "In 3 bullet points: what is a chargeback?"}],
)

# 2) Look inside the response
for block in response.content:
    if block.type == "text":
        print(block.text)

print("\nstop_reason:", response.stop_reason)   # end_turn = finished; tool_use = wants to act (Day 2)
print("blocks:     ", [b.type for b in response.content])
print("tokens:     ", response.usage.input_tokens, "in /", response.usage.output_tokens, "out")

# 3) The model has no memory: WE send the history every time
messages = [{"role": "user", "content": "Remember this merchant id: ACME."}]
r1 = client.messages.create(model=MODEL, max_tokens=1000, messages=messages)

messages.append({"role": "assistant", "content": r1.content})
messages.append({"role": "user", "content": "Which merchant id did I give you?"})
r2 = client.messages.create(model=MODEL, max_tokens=1000, messages=messages)

print("\nmemory test:", [b.text for b in r2.content if b.type == "text"])
```

LAB BEGINNER 15 MIN

Lab 3: Claude vs a local model

Same question to the cloud and to your laptop. Compare quality, speed and cost.

Goal. Run a local open model with Ollama next to Claude and compare the answers.

1. Install Ollama and pull a model

Download from **ollama.com**, then:

RUN
```
ollama pull qwen3:8b
ollama run qwen3:8b "SAY READY"
```

2. Run the side-by-side script

RUN
```
python lab3_side_by_side.py
```

3. Check the arithmetic

Correct answer: (4.2 + 2.0) × 1.10 + 3.1 = **9.92M USD**, which is over the 9.5M limit. Did both models get it right? This is why agents use a **calculator tool** instead of mental maths.

4. Discuss: when would you pick each?

Local: free, private (data stays on the laptop), weaker at reasoning. Cloud: stronger, costs money, data leaves your network. Real systems **route** between them (Day 3).

Full file

`lab3_side_by_side.py`

```python
"""Day 1 – Lab 3: the same question to Claude (cloud) and a local Ollama model.

Prereq: ollama pull qwen3:8b      Run: python lab3_side_by_side.py
"""
import os
import time

import ollama

from llm import MODEL as CLAUDE, client   # Claude via OpenRouter (see llm.py)
LOCAL = os.getenv("OLLAMA_MODEL", "qwen3:8b")

QUESTION = ("A merchant processed 4.2M EUR + 3.1M USD + 2.0M EUR this month. "
            "Their limit is 9.5M USD and 1 EUR = 1.10 USD. Are they over the limit?")

# Claude
start = time.time()
r = client.messages.create(model=CLAUDE, max_tokens=800, messages=[{"role": "user", "content":
QUESTION}])
print(f"--- {CLAUDE} ({time.time() - start:.1f}s) ---")
print("".join(b.text for b in r.content if b.type == "text"))

# Local model
start = time.time()
r = ollama.chat(model=LOCAL, messages=[{"role": "user", "content": QUESTION}])
```

```
print(f"\n--- {LOCAL} ({time.time() - start:.1f}s) ---")
print(r.message.content)

print("\nCorrect answer: (4.2 + 2.0) x 1.10 + 3.1 = 9.92M USD, so YES, over the 9.5M limit.")
```

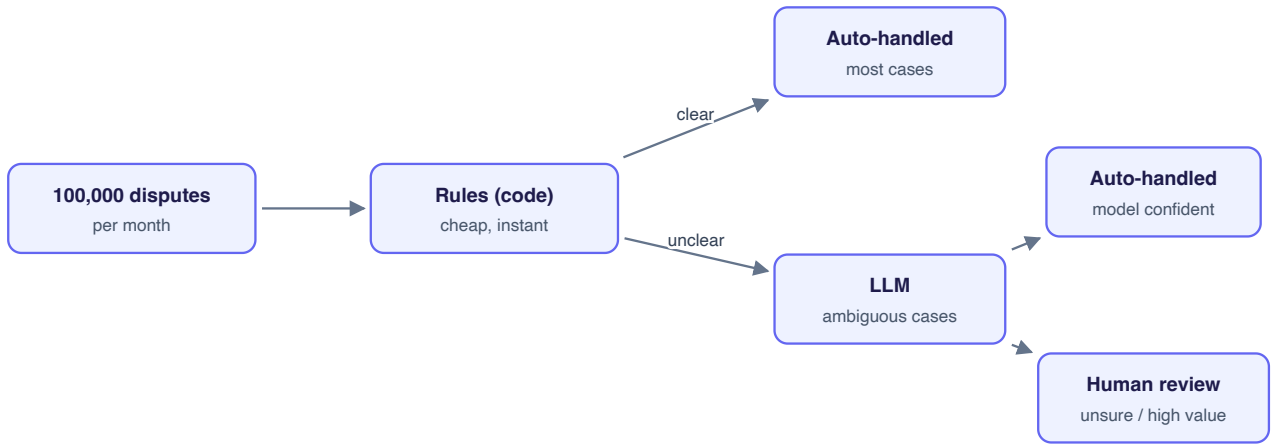
THEORY 10 MIN

Route by uncertainty

The most important cost and reliability idea in agent design.

Use code for the clear, high-volume cases. Use the model only for the genuinely ambiguous ones. Send the model's unsure cases to a human.

Route by uncertainty: chargeback dispute triage



1. Every dispute hits deterministic rules first: known reason code, tiny amount, and so on.
2. Clear cases are handled by code: no tokens, no latency, fully auditable.
3. Only the genuinely ambiguous remainder (messy free-text notes) goes to the model.
4. If the model is confident, it handles the case.
5. If the model is unsure, or the amount is high, a human decides. Measure each route and adjust.

	Rules	LLM
Cost per case	~0	tokens every time
Speed	microseconds	seconds
Audit	exact	probabilistic
Messy free text	❌	✅

Sending **everything** to the model is the most common, and most expensive, beginner mistake.

LAB INTERMEDIATE 15 MIN

Lab 4: Is this even an agent?

A 20-line script that reads a transcript and gives the verdict.

Goal. Tell a plain completion, a single tool call and a real agent loop apart, automatically.

1. Run it on the three sample transcripts

RUN

```
python lab4_is_agent.py transcripts/t1_plain.json
python lab4_is_agent.py transcripts/t2_single_tool.json
python lab4_is_agent.py transcripts/t3_agent_loop.json
```

2. Read t3 and find the moment the observation changed the plan

After `lookup_merchant` returns EUR batches, the next call is `get_fx_rate`. **The observation changed the next action:** that's the loop.

3. Extend it (pick one)

Flag a **stuck** loop (the same call over and over) · flag **sensitive** tools such as `increase_processing_limit` that need approval · find tool calls with no result.

Expected result. t1 → plain completion · t2 → single tool call · t3 → **AGENT**.

Full file

`lab4_is_agent.py`

```
"""Day 1 – Lab 4: is this transcript an agent, or just a chatbot?

Run:  python lab4_is_agent.py transcripts/t3_agent_loop.json
"""
import json
import sys

messages = json.load(open(sys.argv[1]))

tool_calls = []
acting_turns = 0
for m in messages:
    if m["role"] == "assistant" and isinstance(m["content"], list):
        calls = [b for b in m["content"] if b["type"] == "tool_use"]
        tool_calls += calls
        if calls:
            acting_turns += 1

if not tool_calls:
    verdict = "plain completion (no actions)"
elif acting_turns == 1:
    verdict = "single tool call (not a loop yet)"
else:
    verdict = "AGENT: act -> observe -> act again"

print("tool calls:  ", [c["name"] for c in tool_calls])
print("acting turns:", acting_turns)
print("verdict:     ", verdict)
```

QUIZ 5 MIN

Check yourself

1. **What is the architectural difference between generative AI and agentic AI?**
   A. Agents write more fluent text
   B. Agents act, observe the result and decide again, in a loop
   C. Agents use bigger models
   D. Agents don't need prompts

2. **Which are the three building blocks every agent framework uses?**
   A. Prompt, model, output
   B. Tool call, memory, planning
   C. Embeddings, vector store, retriever
   D. Frontend, backend, database

3. **Where on the autonomy spectrum should "raise a merchant's processing limit" sit?**
   A. Fully autonomous
   B. Scripted chain
   C. Human-in-the-loop (at least)
   D. It doesn't matter

4. **In route-by-uncertainty, what handles the high-volume, clear cases?**
   A. The biggest model
   B. A human
   C. Deterministic code (rules)
   D. A local model

5. **A response has stop_reason == "tool_use". What does that mean?**
   A. The model finished
   B. The model is requesting an action: your code must run the tool and return the result
   C. The tool failed
   D. The response was cut off

RECAP 3 MIN

Day 1 recap

- Agentic = closed loop: act → observe → decide again.
- Autonomy is set per action; in payments, money-moving actions need approval.
- Agent = Model + Loop + {Tools, Memory, Planning} + Guardrails.
- Route by uncertainty: rules first, model for the ambiguous rest, humans for the unsure.
- You can call Claude and a local model, and tell a real agent from a chatbot.

Next: Day 2: you'll hand-build Meridian's agent loop from raw API calls: tools, planning, a critic and guardrails, with no framework.

Quiz answer key

1. **B. Agents act, observe the result and decide again, in a loop**
   The defining feature is the closed loop: act → observe → decide again until the goal is met.
2. **B. Tool call, memory, planning**
   Tools (doing), memory (knowing) and planning (sequencing). Frameworks are assemblies of these.
3. **C. Human-in-the-loop (at least)**
   It moves money risk and is hard to undo, so a human approves it. Autonomy is decided per action.
4. **C. Deterministic code (rules)**
   Rules are free, instant and auditable. The model only handles the ambiguous remainder.
5. **B. The model is requesting an action: your code must run the tool and return the result**
   This is the trigger for the agent loop you'll build tomorrow.